Introduction to TrustZone in Cortex-M Microcontrollers

A Leading Provider of Smart, Connected and Secure Embedded Control Solutions

John Haroian

January 26, 2026

Agenda

- Why would a customer need/want TrustZone?
- What is TrustZone?
- Application development for TrustZone

© 2026 Microchip Technology Inc. and its subsidiaries

Why would a customer need/want TrustZone?

MICROCHIP

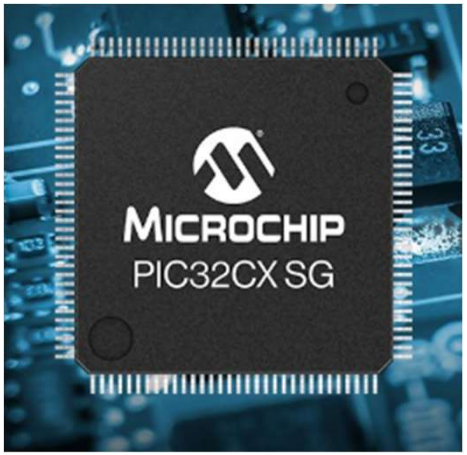
Why would a customer need/want TrustZone

- **Creates additional World within microcontroller**
  - Extends Privilege/Unprivileged
  - One could put independent RTOS in each World

- **System Reliability**
  - Shrink software surface area in Secure World
  - Survive crash on Non-Secure side

- **Extends security model in microcontroller**
  - Secure enclave and Root of trust are possible
  - Why? Portions of the device are isolated from the non-secure world.

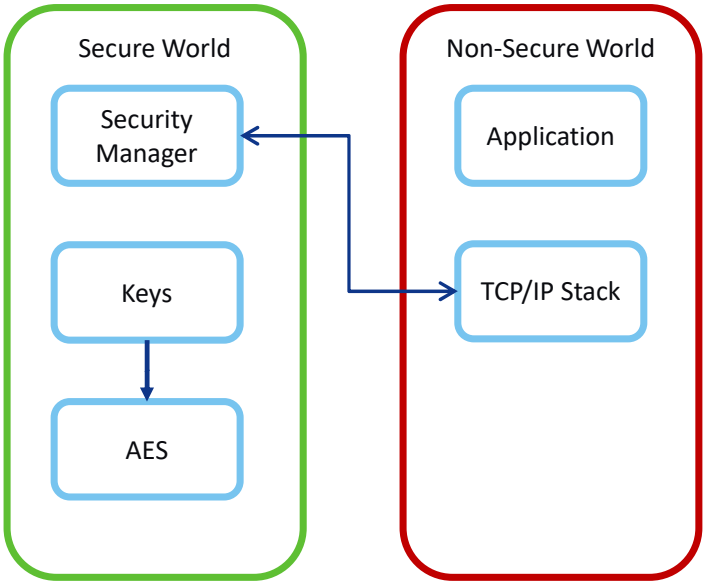
- **IP protection**
  - Isolation and protected entry



© 2026 Microchip Technology Inc. and its subsidiaries

TrustZone Applications

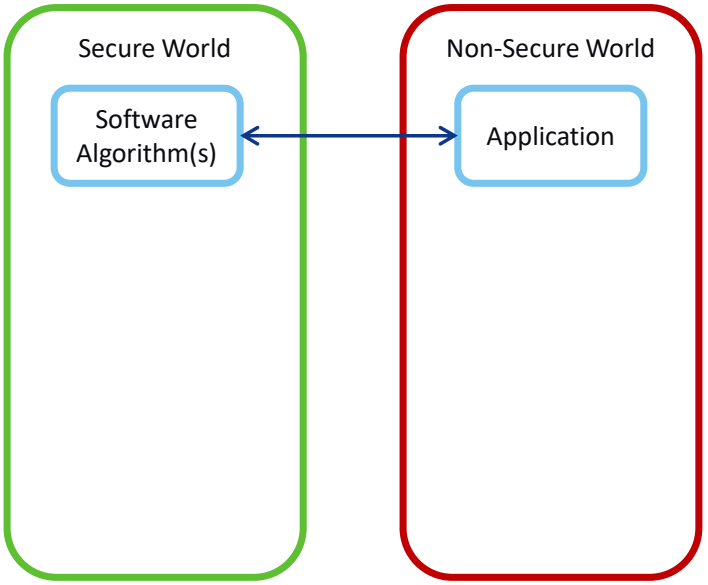
- Data Isolation
  - If the keys are in the Secure World they do not exist in the Non-Secure World
  - TCP/IP Stack (or WolfSSL) not responsible for loading keys into HW AES accelerator
  - Keys that don’t exist can not leak.



© 2026 Microchip Technology Inc. and its subsidiaries

TrustZone Applications

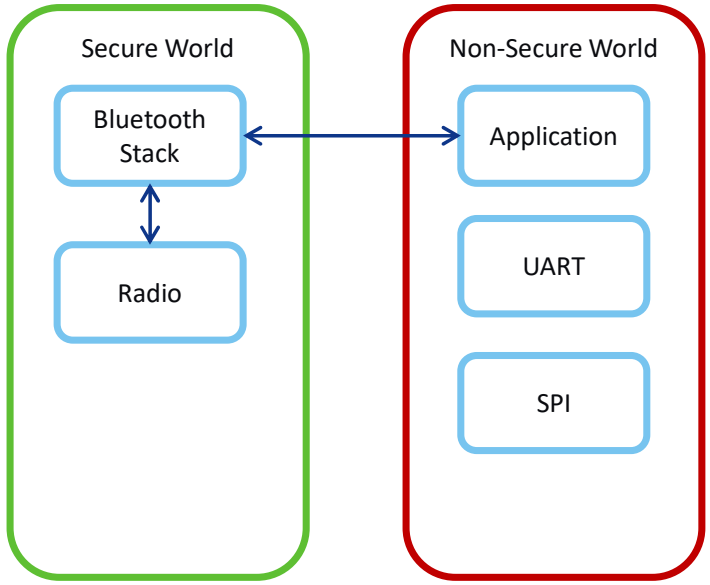
- IP Protection
  - Software can be placed in the Secure World and controlled access granted via API
  - Enables ASSP products
  - Software is protected and controlled.
  - Software Algorithms
    - Certified Stacks
    - Proprietary Methods
    - Edge AI models



© 2026 Microchip Technology Inc. and its subsidiaries

TrustZone Applications

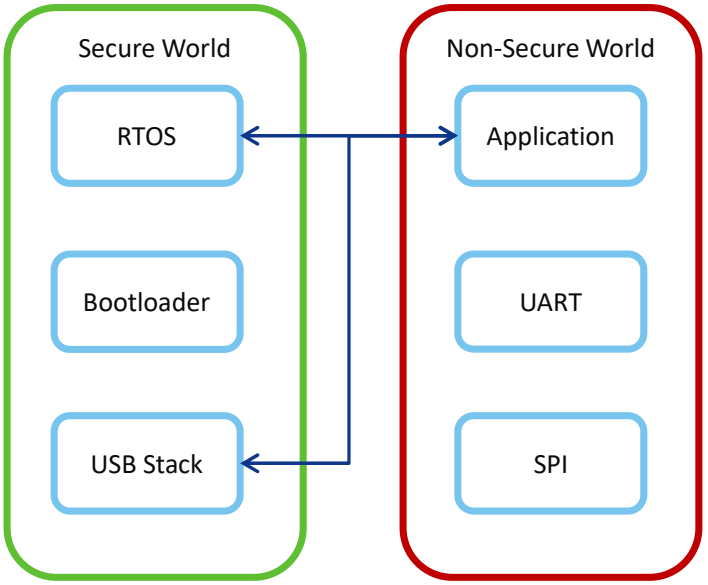
- IP Protection – Wireless Example
  - Bluetooth stack exists in Secure flash. Can be upgraded securely
  - API is published and controlled
  - User application can exist fully
    - Peripherals
    - Interrupts
    - Updates



© 2026 Microchip Technology Inc. and its subsidiaries

TrustZone Applications

- IP Protection – ASSP
  - Limited by customer's creativity
  - Consider ease of development



© 2026 Microchip Technology Inc. and its subsidiaries

Objections

- Seems complicated/hard
- Understood. Implementing is intentional and requires effort
- No one wants to break my application
- Well, if it has no value then why are you doing it?
- Where is the path to compromise?
- I'm glad you asked…

© 2026 Microchip Technology Inc. and its subsidiaries

Hacking Washing Machines

- Access at:
  - https://media.ccc.de/v/39c3-hacking-washing-machines

- Optical interface to UART

- Brute forced the communication protocol

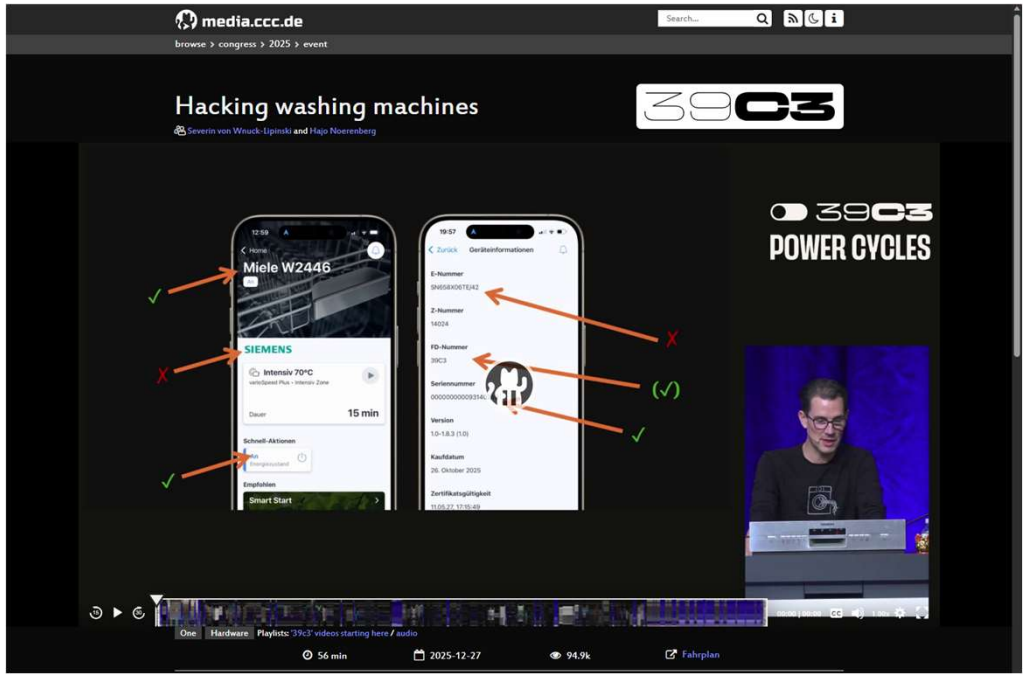
- Used timing attack (oscilloscope)

- After unlocked, diagnostic commands included read memory and EEPROM

- Why?

- They wanted to update status and control appliance from Home Assistant

- **Deeply embedded device with questionable value in hacking the firmware.**



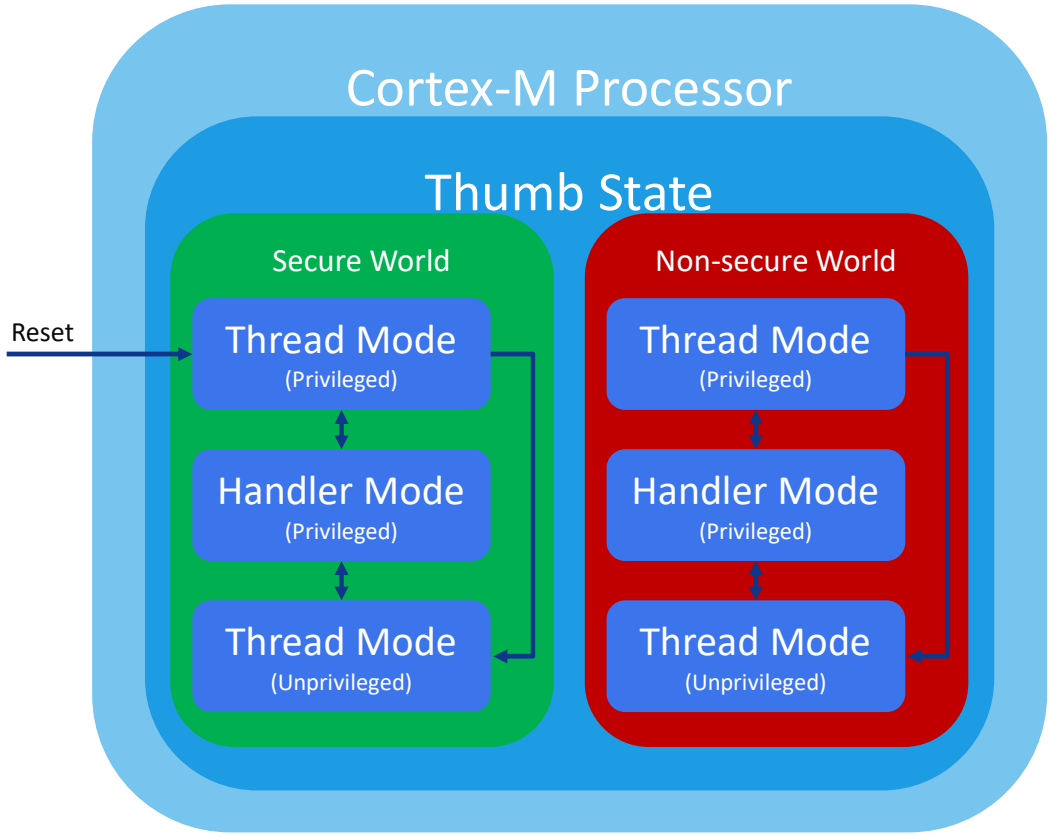
© 2026 Microchip Technology Inc. and its subsidiaries

What is TrustZone?

MICROCHIP

What is TrustZone?

- Added in ARMv8-M ISA
  - Cortex-M23 and Cortex-M33
- Optional feature
  - PIC32CM LE vs LS
- Security feature?
- Isolation feature via Worlds



© 2026 Microchip Technology Inc. and its subsidiaries

SAMD21 Memory Map

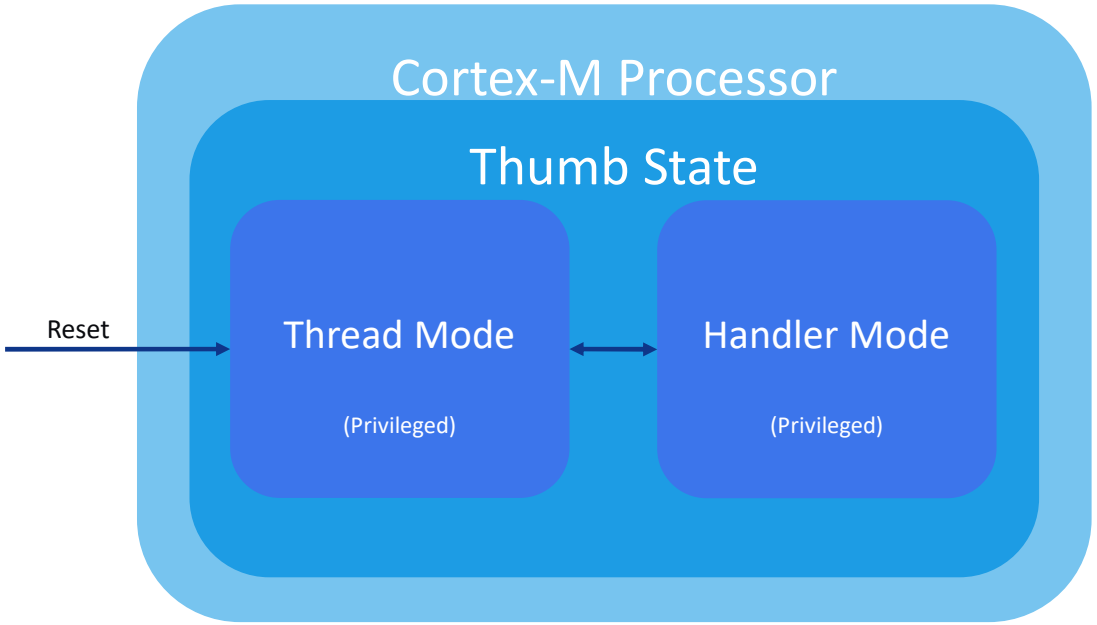
- Cortex-M0+ - ARMv6-M ISA
- 32 bit Micro = 4GB Address Space
- Unfettered access to entire space
- Our “normal” mental model of embedded programming

Global Memory Space	
0x0000 0000	Code
0x2000 0000	SRAM
0x2000 8000	*Undefined*
0x4000 0000	Peripherals
0x4300 0000	*Reserved*
0x6000 0000	IOBUS
0x6000 0200	*Reserved*
0xE000 0000 0xFFFF FFFF	System

© 2026 Microchip Technology Inc. and its subsidiaries

SAMD21 Execution Model

- Two modes
  - Thread – application code
  - Handler – exception code



- No Memory Protection Unit (MPU) providing memory attribute control

- All code can access the entire memory map

Most Bare-Metal Embedded Applications

© 2026 Microchip Technology Inc. and its subsidiaries

What is a MPU?

- **Memory Protection Unit**
  - Peripheral found in Cortex-M devices
  - Lite version of a Memory Management Unit (MMU) found in Cortex-A devices
  - Provides dynamic, programmable memory access rules (attributes)
    - Read, Write, Execute…
    - Cacheable, Shareable
    - Privileged or Unprivileged
  - Violation = Hard Exception
  - Limited resource – 8 or 16 regions

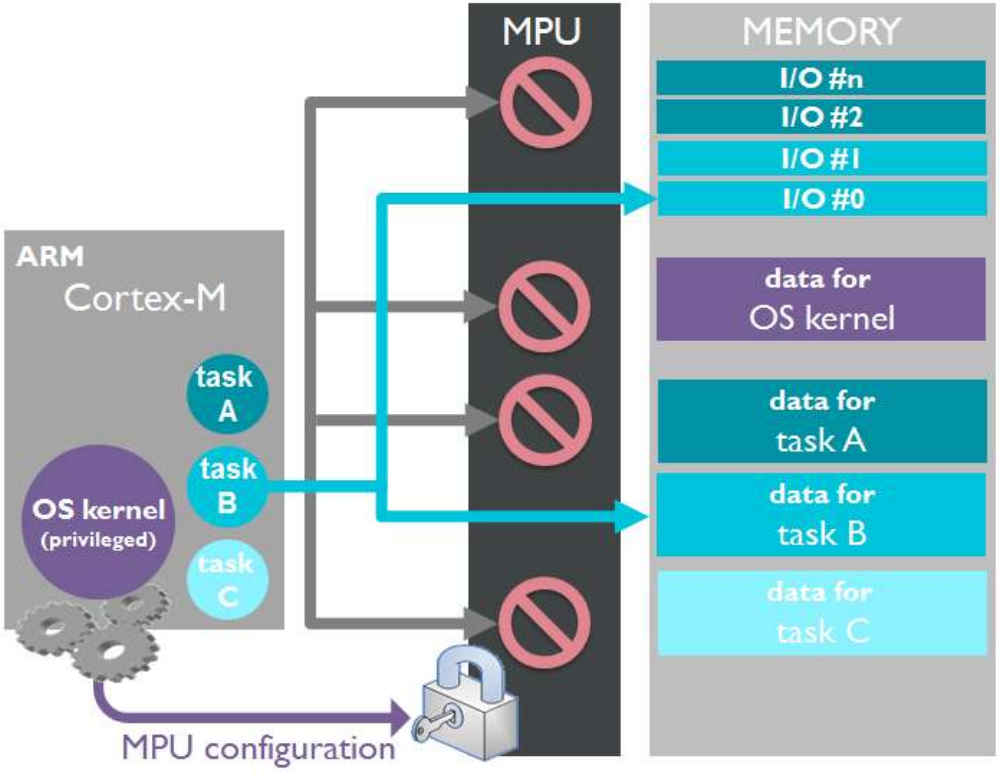
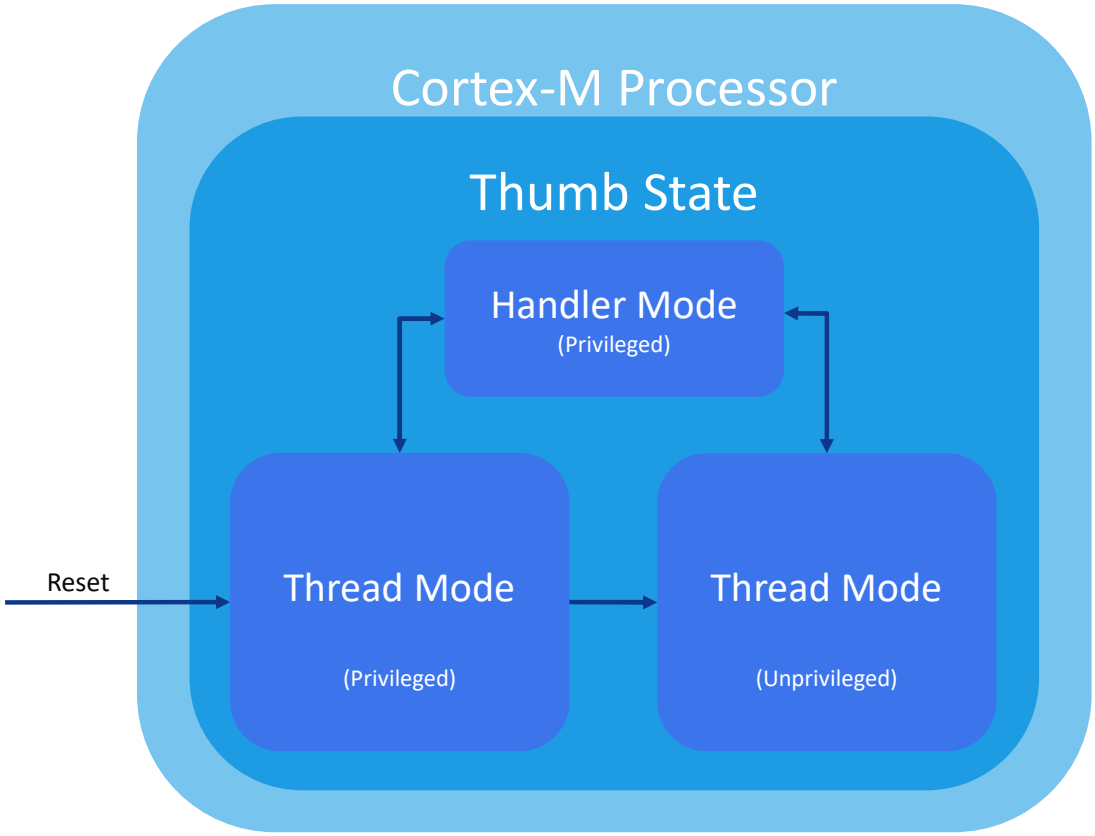


Image from ARM website

© 2026 Microchip Technology Inc. and its subsidiaries

SAMC21 Execution Model

- Two access levels
  - Privileged and Unprivileged
- Transitions between access levels controlled
  - Handler Mode is always privileged
  - Unprivileged transitions via exception
- Separate Stack pointers and MPU controls memory access
- Unprivileged cannot access privileged resources (memory)



© 2026 Microchip Technology Inc. and its subsidiaries

Where are access levels used?

- RTOS and Functional Safety
  - Protect the kernel from being crashed by a task
  - Isolate and enforce memory areas
  - Prevent changes to hardware
- Support access levels
  - FreeRTOS - Memory Protection Unit (MPU) Support - FreeRTOS™
  - Zephyr - Arm Cortex-M Developer Guide — Zephyr Project Documentation
  - CMSIS-RTOS2 - CMSIS-RTOS2: MPU Protected Zones
- Must be configured and intentionally used

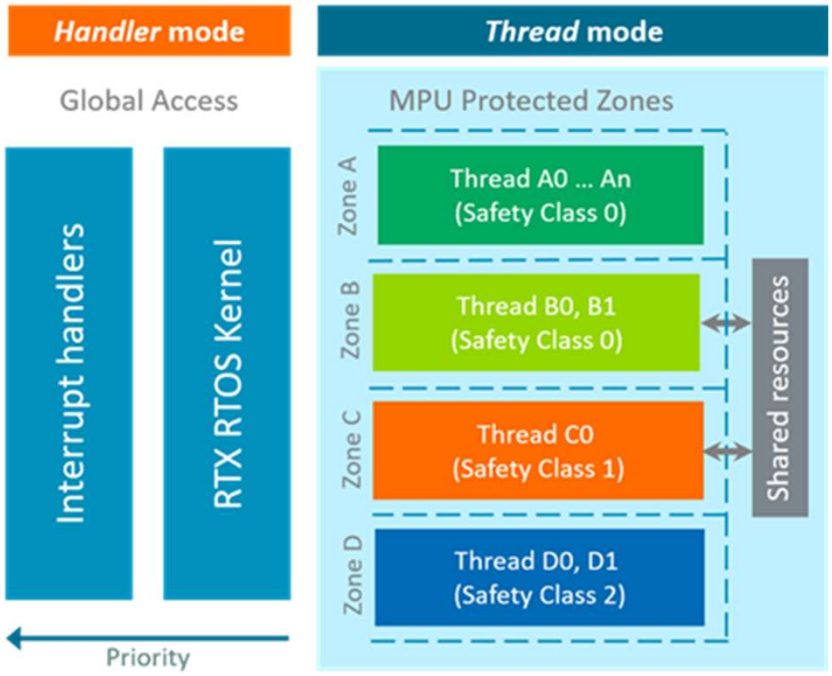


Image from ARM website

© 2026 Microchip Technology Inc. and its subsidiaries

PIC32CM LS00 Memory Map

- Cortex-M23 – ARMv8-M ISA
- 32 bit Micro = 4GB Address Space
- IDAU adds additional attributes
  - Secure (S)
  - Non-Secure (NS)
  - Non-Secure Callable (NSC)
  - Exempt
- Exempt is used for:
  - Peripherals
  - IOBUS
  - Certain NVM Rows
  - Security is done at the peripheral level

Global Memory Space			
0x0000 0000 - Flash	BOOTPROT		Boot Secure
		BNSC	Boot Non-Secure Callable
	AS		Application Secure
		ANSC	Application Non-Secure Callable
			Application Non-Secure
0x0040 0000 – Data Flash	DS		Data Secure
			Data Non-Secure
0x0200 0000 – Boot ROM			Secure
0x2000 0000 - SRAM	RS		Secure
			Non-Secure
0x4000 0000 - Peripherals			Exempt
0x6000 0000 - IOBUS			Exempt
0xE000 0000 – System/PPB 0xFFFF FFFF			Secure

© 2026 Microchip Technology Inc. and its subsidiaries

SAU vs IDAU

- **SAU – Security Attribution Unit**
  - Dynamic and changeable at run-time (Secure only)
  - Microchip does not use
- **IDAU – Implementation Defined Attribution Unit**
  - Fixed at run-time
  - Modified via User Configuration row (UROW) and Boot Configuration row (BOCOR)
  - Used in all Microchip Trust Zone implementations
- **Both are used to define several areas of memory in the memory map**

Field	Bit Width	Size
AS (App)	10	AS*256
ANSC	8	ANSC*32
DS (Data)	6	DS*32
RS (SRAM)	8	RS*128
BOOTPROT	10	BOOTPROT*256
BNSC	8	BNSC*32

© 2026 Microchip Technology Inc. and its subsidiaries

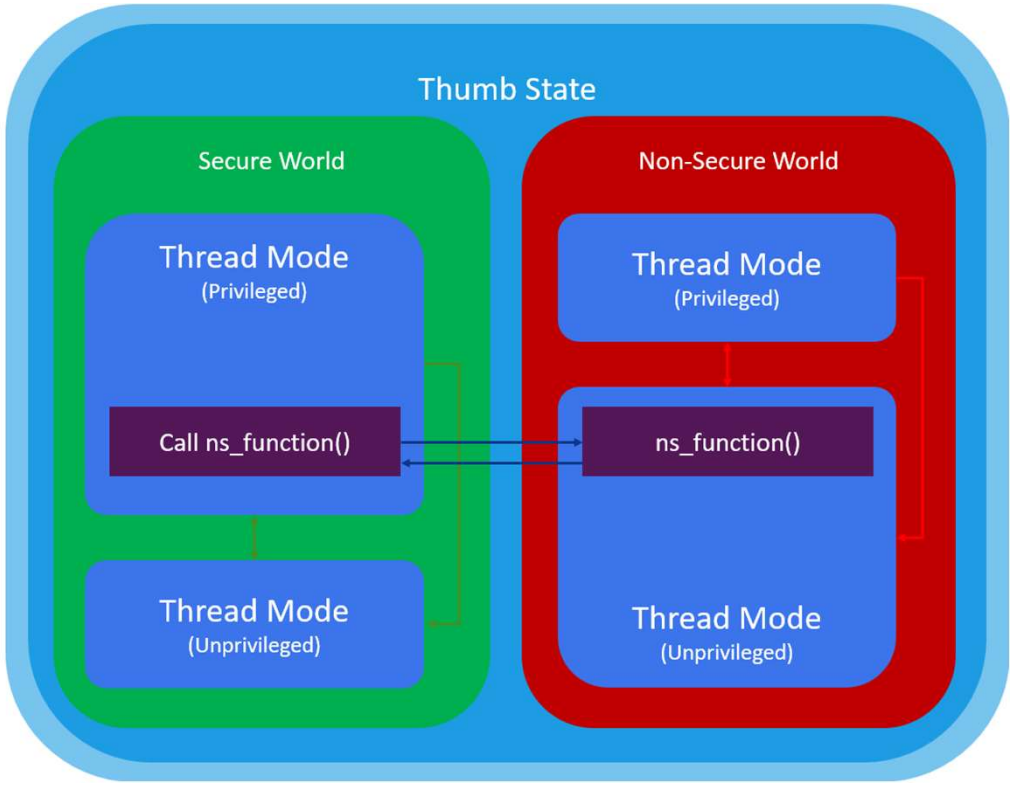
Duplicated Peripherals

- To support flexible development, the following peripherals are duplicated:
  - MPU
  - NVIC
  - SysTick
- Enables support for a RTOS in each World
- Also enables interrupt support in both worlds (separate vector tables and NVIC)
- Core timer available in both

© 2026 Microchip Technology Inc. and its subsidiaries

Moving between Worlds

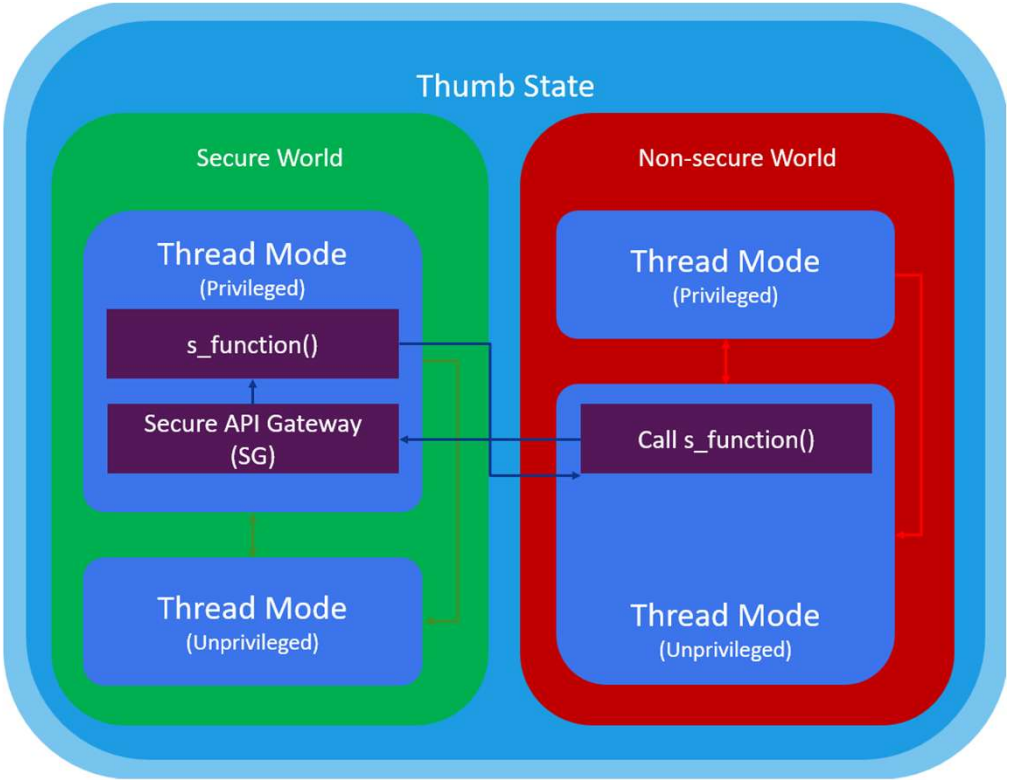
- **Secure -> Non-Secure**
  - Easy - function call



© 2026 Microchip Technology Inc. and its subsidiaries

Moving between Worlds

- **Non-Secure -> Secure**
  - More difficult or controlled
  - Secure API (function or veneer) must reside in the Non-Secure Callable area (NSC)
  - First instruction of veneer must be SG (placed by ARM extension to complier or –mcmse option)
  - This allows for the s_function (located in Secure area) to be called without a Hard Fault.



© 2026 Microchip Technology Inc. and its subsidiaries

Application development for TrustZone

MICROCHIP

Microchip Micros based on ARMv8-M

- ARMv8-M Baseline
  - SAML10/11
  - PIC32CM LE/LS
  - *PIC32CM GC/SG*
  - *PIC32CM PC*
- ARMv8-M Main
  - PIC32CK GC/SG
  - *PIC32CK MC*
- Cortex-M23 Core with TrustZone
  - SAML11
  - PIC32CM LS00/60
- Cortex-M33 Core with TrustZone
  - PIC32CK SG

© 2026 Microchip Technology Inc. and its subsidiaries

Methods and Models

- Microchip currently supports only projects containing Secure and Non-Secure
  - MPLAB X
  - Scenarios #2-4 with #2 nominal
- Extensions for VSCode support coming soon
  - Will support separation of secure and non-secure projects
  - All scenarios
- Scenario #5 is a boot loader under our current model

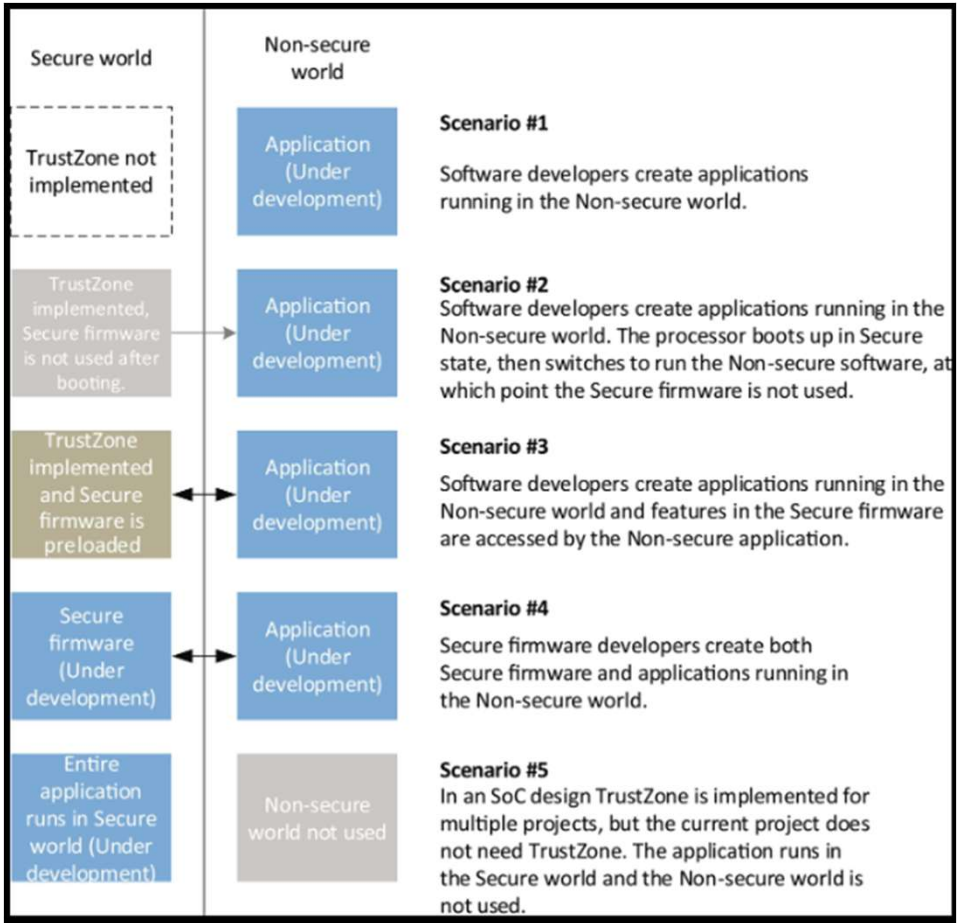


FIG. 17.1 Software development scenarios in Armv8-M processors.

© 2026 Microchip Technology Inc. and its subsidiaries

Demos

- **Step 01**
  - Secure only
  - Disabled template code calling non-secure application
  - Blink S_LED0 with Secure SysTick timer for delay
  - Printf via Secure UART
- **Step 02**
  - Add non-secure code
  - Re-enable template to call non-secure application
  - Blink NS_LED1 with SysTick timer
  - Add Non-Secure UART
- **Step 03**
  - Allow Secure code to run via interrupt (Secure SysTick)
  - Blink both LEDs
  - Messages on both UARTs
- **Step 04**
  - Demonstrate Non-Secure application calling Secure function
- **Step 05**
  - Demonstrate Non-Secure application attempt to access Secure memory

© 2026 Microchip Technology Inc. and its subsidiaries

Resources

- **Definitive Guide to ARM Cortex-M23 and Cortex-M33 Processors** by Joseph Yui

- **ARMv8-M Techncial Overview Whitepaper** by Joseph Yui

- **Hacking Washing Machines** by Severin von Wnuck-Lipinski and Hajo Noerenberg

© 2026 Microchip Technology Inc. and its subsidiaries